

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I K PHANI SRIDHAR YOGI/ 192311091 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity¹	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using **unsalted SHA-1 hashes**. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability.

Recommend a secure password hashing strategy and justify your choice.

Solution :

Weak Password Hashing in Web Application

Understanding of the Case

A web application stores user passwords using **unsalted SHA-1 hashes**. After the database is compromised, attackers use **precomputed rainbow tables** to recover many user passwords quickly. Since identical passwords generate identical hashes and SHA-1 is computationally fast, the attackers can crack passwords efficiently.

Application of Concepts

SHA-1

SHA-1 is a cryptographic hash function that produces a **160-bit hash value**. Although once widely used, it is now considered insecure because:

- It is vulnerable to collision attacks.
- It is very fast, making brute-force attacks practical.
- It is no longer recommended for password storage.

Salting

A **salt** is a unique random value added to each password before hashing.

Example:

Password:

Password123

Without Salt:

SHA1>Password123)

With Salt:

SHA1>Password123 + X7A91K)

Even if two users choose the same password, their hashes become different.

Analysis

Why the System Was Vulnerable

1. No Salt

- Identical passwords produced identical hashes.
 - Rainbow tables could crack passwords instantly.
2. Use of SHA-1
- Extremely fast hashing.
 - Allows attackers to test millions of passwords every second.
3. Lack of Key Stretching
- Only one hashing iteration.
 - Makes brute-force attacks inexpensive.

Consequences:

- User accounts compromised.
- Credential reuse attacks.
- Loss of customer trust.

Solution Design

Recommended Password Hashing Strategy

Use **Argon2** (recommended), **bcrypt**, or **PBKDF2**.

Recommended:

- Argon2id
- Random 16-byte salt
- Memory-hard computation
- High iteration count

Benefits:

- Protects against rainbow table attacks.
- Slows brute-force attacks.
- Each password has a unique hash.
- Resistant to GPU-based cracking.

Justification

Argon2 is the current password hashing standard because it combines **salting**, **memory hardness**, and **key stretching**, making password cracking significantly more difficult than SHA-1.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose

mitigation strategies to secure private keys and prevent such incidents.

Solution :

Digital Signature Compromise in Financial System

Understanding of the Case

A financial institution uses digital signatures to authorize transactions. An attacker steals a user's private key and signs fraudulent transactions, making them appear legitimate.

Application of Concepts

Digital signatures use:

- Private Key → Signing
- Public Key → Verification

Security properties:

- Authentication
- Integrity
- Non-repudiation

Analysis

Effect on Authentication

The attacker possesses the legitimate private key.

Result:

- Fake transactions appear genuine.

Authentication fails because:

- The system cannot distinguish the attacker from the legitimate user.

Effect on Non-Repudiation

Normally:

User → Signs transaction → Cannot deny it later.

After key compromise:

- The user may deny the transaction.
- The bank cannot determine who actually signed it.

Non-repudiation is weakened.

Solution Design

Mitigation:

1. Store private keys in Hardware Security Modules (HSMs).
2. Use smart cards or USB security tokens.
3. Enable Multi-Factor Authentication (MFA).

4. Rotate keys periodically.
5. Use certificate revocation immediately after compromise.
6. Monitor unusual transaction behavior.
7. Protect keys using encrypted key stores.

Justification

Combining HSMs, MFA, and certificate revocation significantly reduces the likelihood and impact of private key compromise while preserving authentication and non-repudiation.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

Solution :

Kerberos Authentication Failure in Distributed System

Understanding of the Case

A distributed system using Kerberos experiences authentication failures because the clocks on different servers are not synchronized.

Application of Concepts

Kerberos relies on:

- Tickets
- Timestamps
- Ticket Granting Server (TGS)

Tickets have limited validity.

Analysis

If clocks differ significantly:

Example

Client:

10:00 AM

Server:

9:54 AM

Kerberos assumes the ticket is invalid because the timestamp falls outside the acceptable time window.

Effects:

- Login failures.
- Ticket rejection.
- Interrupted services.

Solution Design

Recommended solutions:

1. Use Network Time Protocol (NTP).
2. Synchronize all servers with a common time server.
3. Configure redundant NTP servers.
4. Allow a small clock skew (typically 5 minutes).
5. Continuously monitor system clocks.

Justification

Time synchronization ensures Kerberos tickets remain valid while still protecting against replay attacks, improving authentication reliability.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

Solution :

Certificate Authority Breach in PKI System

Understanding of the Case

A trusted Certificate Authority (CA) is compromised and issues fraudulent X.509 certificates for legitimate websites.

Application of Concepts

Public Key Infrastructure (PKI) depends on:

- Root CAs
- Intermediate CAs
- Digital certificates
- Certificate chains

Browsers trust certificates signed by trusted CAs.

Analysis

Consequences:

- Fake websites appear legitimate.
- HTTPS connections become vulnerable.
- Man-in-the-Middle attacks become possible.
- Users unknowingly reveal sensitive information.
- Trust in the PKI ecosystem is reduced.

Solution Design

Recommended mechanisms:

1. Certificate Revocation Lists (CRLs).
2. Online Certificate Status Protocol (OCSP).
3. Certificate Transparency (CT) logs.
4. Short-lived certificates.
5. Hardware Security Modules for CA private keys.
6. Multi-party approval for certificate issuance.
7. Rapid revocation of compromised certificates.

Justification

Certificate Transparency enables unauthorized certificates to be detected quickly, while CRLs and OCSP prevent clients from trusting revoked certificates. Securing CA private keys with HSMs reduces the risk of compromise.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures.

Recommend a robust integrity verification mechanism and justify its effectiveness.

Solution :

Integrity Verification in Cloud Data Storage

Understanding of the Case

A cloud provider uses hashing to verify file integrity, but improper validation allows modified files to go undetected.

Application of Concepts

Hashing verifies integrity by comparing:

- Original hash
- Current hash

If both match:

- File is unchanged.

If different:

- File has been altered.

Analysis

Possible implementation mistakes:

- Comparing incorrect hash values.
- Using weak algorithms (MD5 or SHA-1).
- Not securely storing the original hash.
- Ignoring failed hash comparisons.
- Not validating files after transmission.

Consequences:

- Data tampering remains undetected.
- Loss of trust.
- Possible corruption of critical data.

Solution Design

Recommended approach:

1. Use SHA-256 or SHA-3.
2. Use HMAC-SHA256 for authenticated integrity verification.
3. Digitally sign important files.
4. Store hashes securely.
5. Perform integrity checks during upload and download.
6. Use Merkle Trees for large datasets.
7. Maintain integrity audit logs.

Justification

HMAC-SHA256 provides stronger protection than simple hashing because it uses a secret key, preventing attackers from generating valid hashes for tampered data. Digital signatures further provide authentication and non-repudiation, making the integrity verification process more secure.